

FixForesight

Person B — Backend, Frontend & Infra Build Guide

1. What This Project Is

FixForesight is an industrial-grade Predictive + Prescriptive Maintenance platform. It combines Big Data processing, Machine Learning, and Cloud infrastructure to solve a real industrial problem: predicting when a machine will fail, and recommending exactly what should be done about it.

The system has two ML-driven capabilities working together:

- **Predictive layer:** IoT sensor data is streamed in, processed at scale with Apache Spark, and fed into a TensorFlow model that predicts the probability and type of an upcoming machine failure.
- **Prescriptive layer (Recommendation Engine):** once a failure is predicted, a recommendation model suggests the best corrective action, the spare parts likely needed, and how urgently it should be scheduled relative to other flagged machines.

All of this is built on an AWS-equivalent cloud architecture (S3, SQS, SNS, RDS — simulated locally via LocalStack at zero cost) and exposed through a backend API and a live React/Redux dashboard.

End-to-End Data Flow

Sensor Simulator → SQS → Spark (feature engineering) → S3 + Postgres → TensorFlow failure model + Recommendation model → Postgres (predictions & recommendations) → SNS alerts + FastAPI backend → React/Redux dashboard. Solr indexes historical incidents for search.

2. Team Split

Person A (Teammate)	Person B (You)
Sensor simulator	FastAPI backend
Spark feature engineering	React + Redux frontend
TensorFlow failure model	Solr integration
Recommendation model	LocalStack S3/SQS/SNS wiring (API side)
Owns Postgres schema (shared doc)	Consumes Postgres schema (shared doc)

Why this split: you can build the entire backend and frontend against mock/sample data while your teammate trains models — neither of you blocks the other.

3. Your Step-by-Step Plan (Person B)

Step 1 — Agree on Contracts (Day 1, do this together)

Before writing code, sit with your teammate and finalize `CONTRACTS.md`: the exact Postgres schema (table/column names and types) and the exact API request/response JSON shapes. This is the single most important step — it lets you both build independently without breaking each other.

Step 2 — Set Up Your Local Environment

Install Docker, Docker Compose, Node.js 18+, Python 3.10+, and AWS CLI. Clone the repo. Run 'docker compose up -d postgres localstack solr' and confirm all three show healthy via 'docker compose ps'.

Step 3 — Build the Postgres Schema (jointly owned)

Apply schema.sql to Postgres. Confirm tables exist: sensor_readings, work_orders, parts_inventory, predictions, recommendations. Run 'docker exec -it postgres psql -U postgres -d pdm_db -c "\dt"'.

Step 4 — Create Mock Data So You're Not Blocked

While Person A trains the models, write a seed.sql or a small Python script that inserts fake rows into predictions and recommendations tables matching the agreed contract. This unblocks your entire backend and frontend build before the real models exist.

Step 5 — Build the FastAPI Backend

Create endpoints: GET /machines/{id}/risk, GET /machines/{id}/recommendations, GET /alerts, GET /health. Connect to Postgres using the Docker service name 'postgres' as host (not localhost). Add CORSMiddleware allowing http://localhost:3000 so the frontend can call it without errors.

Step 6 — Wire Up LocalStack on the API Side

Add SNS subscription handling so the backend can receive/relay alerts, and SQS read access if the API needs to check on job status. Use endpoint_url=http://localstack:4566 inside containers.

Step 7 — Containerize the Backend

Write a Dockerfile for the backend, build it, and run it via docker compose. Test with 'curl http://localhost:8000/health' and 'curl http://localhost:8000/machines/1/risk'.

Step 8 — Build the React + Redux Frontend

Set up Redux slices for: machine risk data, recommendations, and alerts. Build dashboard components: a machine risk list/heatmap, a detail view showing predicted failure + recommended action + parts needed, and an alerts feed. Use Webpack (via Create React App or custom config) to bundle for production.

Step 9 — Integrate Solr

Stand up the Solr container, define a schema for historical incidents (machine_id, failure_signature, action_taken, outcome, date), index sample data, and add a search bar in the frontend that queries Solr through a backend endpoint (e.g., GET /incidents/search?q=...).

Step 10 — Connect Real Model Outputs

Once Person A's models are producing real predictions and recommendations into Postgres, swap your mock data for the real tables. Run an end-to-end test together: simulator running, Spark job run, dashboard showing live, real predictions.

Step 11 — Polish and Document

Add loading states and error handling in the frontend, write the README with architecture diagram and setup instructions, and validate your CloudFormation/Ansible files reflect what was actually built.

4. How You'll Know You're On Track

- Backend runs standalone and returns valid JSON from all endpoints, even with mock data.
- Frontend renders real data fetched from your backend, with no CORS errors in the browser console.
- Solr search returns relevant historical incidents from the dashboard search bar.
- After Step 10, the dashboard reflects real model predictions with no mock data left in the code path.
- `docker compose ps` shows every one of your services as 'Up (healthy)'.

5. Golden Rule for Working in Parallel

Never let the other person discover a broken contract by accident. If you change an API response shape, a table column, or an endpoint name — message your teammate immediately. This single habit prevents almost all the wasted time in two-person builds.